

目录

CSRF 漏洞修复报告 1

目录 1

1. 漏洞概述 2

2. 漏洞一：修改密码无 CSRF 保护 2

2.1 问题描述 2

2.2 问题代码 2

2.3 攻击示例 3

2.4 危害等级 3

3. 漏洞二：充值功能无 CSRF 保护 3

3.1 问题描述 3

3.2 攻击示例 4

4. CSRF 漏洞原理 4

4.1 什么是 CSRF 4

4.2 攻击原理图 4

4.3 CSRF 攻击成功的三个条件 4

4.4 为什么传统的防范措施不够 4

5. 修复方案 5

5.1 修复对照表 5

5.2 修复后核心代码 5

5.3 CSRF Token 的工作原理 6

6. 修复前后代码对比 6

后端对比 6

前端对比 7

7. 安全建议 7

7.1 CSRF 防御措施优先级 7

7.2 Flask 开发者 CSRF 检查清单 7

7.3 关键操作额外验证 8

7.4 完整的 CSRF 防御架构 8

附录：OWASP CSRF 防御指南 8

CSRF 漏洞修复报告

项目名称：用户信息管理系统
报告日期：2026-07-13
审计目标：Flask 用户管理平台中的 CSRF（跨站请求伪造）漏洞分析与修复
报告人：安全审计组

目录

1. 漏洞概述

2. 漏洞一：修改密码无 CSRF 保护
3. 漏洞二：充值功能无 CSRF 保护
4. CSRF 漏洞原理
5. 修复方案
6. 修复前后代码对比
7. 安全建议

1. 漏洞概述

在本次安全审计中，发现目标系统的修改密码和充值功能存在 **2 项 CSRF 漏洞**，攻击者可构造恶意页面，诱导已登录用户在不知情的情况下执行敏感操作。

编号	漏洞名称	风险等级	攻击入口
CSRF-001	修改密码无 CSRF 保护	高危	/change-password POST 接口
CSRF-002	充值功能无 CSRF 保护	中危	/recharge POST 接口

2. 漏洞一：修改密码无 CSRF 保护

2.1 问题描述

修改密码接口没有任何 CSRF 保护机制，攻击者可构造恶意 HTML 页面，受害者只要访问该页面就会自动触发密码修改请求，且无需知道原密码。

2.2 问题代码

```
@app.route("/change-password", methods=["POST"])
def change_password():
    # 只要 session 中有登录状态即可
    if "username" not in session:
        return redirect("/login")

    # 直接从表单接收参数，不验证来源、不验证原密码
    target_username = request.form.get("username")
    new_password = request.form.get("new_password")

    # 直接更新密码
    USERS[target_username]["password"] = new_password
```

2.3 攻击示例

攻击者构造一个恶意 HTML 页面：

```
<html>
<body>
<h1>点击领取免费皮肤! </h1>
<form id="csrf_form" action="http://192.168.131.129:5000/change-password" method="POST">
  <input type="hidden" name="username" value="admin">
  <input type="hidden" name="new_password" value="hacked_by_csrf">
</form>
<script>
  // 页面加载后自动提交表单
  document.getElementById("csrf_form").submit();
</script>
</body>
</html>
```

攻击流程：

1. 攻击者制作恶意页面 → 托管在攻击者服务器上
2. 诱骗管理员访问该页面
3. 管理员浏览器自动发起 POST 请求到 /change-password
4. 管理员的 session cookie 自动携带 (SameSite 未设置)
5. 服务器验证 session 通过 → 密码被修改为 hacked_by_csrf
6. 攻击者使用新密码登录管理员账号

2.4 危害等级

攻击类型	可能性	影响
账户劫持	高	任意用户密码可被修改
权限提升	高	劫持管理员账户
CSRF 蠕虫	中	自动化传播攻击

3. 漏洞二：充值功能无 CSRF 保护

3.1 问题描述

充值接口同样没有 CSRF 保护，攻击者可构造恶意页面让受害者自动给攻击者的账户充值。

3.2 攻击示例

```
<form action="http://192.168.131.129:5000/recharge" method="POST">
  <input type="hidden" name="amount" value="-9999">
  <input type="hidden" name="user_id" value=" 受害者的 ID">
</form>
<script>document.forms[0].submit()</script>
```

4. CSRF 漏洞原理

4.1 什么是 CSRF

CSRF (Cross-Site Request Forgery, 跨站请求伪造) 是一种攻击, 诱导用户在已登录的状态下, 向目标网站发送恶意请求。

4.2 攻击原理图

受害者浏览器 (已登录目标网站)

访问攻击者的恶意页面

恶意页面自动提交表单到目标网站

浏览器自动携带目标网站的 Cookie (session)

服务器接收请求 → 验证通过 (Cookie 有效)
→ 执行操作 (密码被修改)
→ 攻击者成功!

4.3 CSRF 攻击成功的三个条件

条件	说明	本项目中
① 用户已登录目标网站	session cookie 有效	admin 已登录
② 目标网站有可被利用的操作接口	POST 接口无 CSRF 保护	/change-password 无 Token 验证
③ 接口不验证请求来源	没有检查 Referer 或 Origin	未做任何来源检查

4.4 为什么传统的防范措施不够

措施	为什么不够
仅依靠 Cookie SameSite 检查 Referer 头 验证码	部分浏览器不支持，旧版本兼容性问题 部分场景下 Referer 可能被禁用 用户体验差，不能覆盖所有操作

5. 修复方案

5.1 修复对照表

漏洞编号	修复措施	风险等级变化
CSRF-001	集成 Flask-WTF CSRF 保护，所有 POST 表单添加 csrf_token	高危 → 已修复
CSRF-002	充值表单同样添加 CSRF Token	中危 → 已修复

5.2 修复后核心代码

```
from flask_wtf.csrf import CSRFProtect

csrf = CSRFProtect(app)  # 全局启用 CSRF 保护
```

后端：启用全局 CSRF 保护 Flask-WTF 的 CSRFProtect 会自动拦截所有 POST 请求，验证请求体中是否包含有效的 csrf_token。只需要一行代码，所有 POST 接口自动获得 CSRF 保护。

```
<form method="POST" action="/change-password">
  <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
  <input type="hidden" name="username" value="{{ user['username'] }}">
  <input type="password" name="old_password" placeholder="原密码" required>
  <input type="password" name="new_password" placeholder="新密码" required>
  <button type="submit">修改密码</button>
</form>
```

前端：在表单中添加 CSRF Token {{ csrf_token() }} 是 Flask-WTF 提供的模板函数，它会在页面中生成一个隐藏字段，包含服务端生成的随机 Token。提交表单时，服务器会验证这个 Token 是否有效。

5.3 CSRF Token 的工作原理

用户请求页面 (GET)

服务器生成一个随机 Token
存入 Session
写入页面中的隐藏字段 csrf_token

用户提交表单 (POST)

浏览器携带 csrf_token 和 session cookie

服务器验证 csrf_token 是否与 Session 中的匹配
匹配 → 处理请求
不匹配 → 拒绝请求 (400 Bad Request)

攻击者无法获取受害者的 CSRF Token，因为：

1. Token 在页面中，跨域无法读取（同源策略）
2. Token 每次会话不同，不可预测
3. Token 与 Session 绑定，无法伪造

6. 修复前后代码对比

后端对比

```
# ===== 漏洞版 (无 CSRF 保护) =====
app = Flask(__name__)
app.secret_key = "dev-key-2025"

@app.route("/change-password", methods=["POST"])
def change_password():
    if "username" in session:
        # 直接处理请求，不验证 CSRF Token
        ...

# ===== 修复版 (全局 CSRF 保护) =====
from flask_wtf.csrf import CSRFProtect

app = Flask(__name__)
app.secret_key = os.environ.get("SECRET_KEY", os.urandom(32).hex())

csrf = CSRFProtect(app) # ← 一行代码启用全局 CSRF 保护
```

```
@app.route("/change-password", methods=["POST"])
def change_password():
    # CSRFProtect 自动拦截没有有效 Token 的 POST 请求
    ...
```

前端对比

```
<!-- ===== 漏洞版 (无 CSRF Token) ===== -->
<form method="POST" action="/change-password">
    <input type="hidden" name="username" value="admin">
    <input type="password" name="new_password">
    <button type="submit">修改密码</button>
</form>

<!-- ===== 修复版 (添加 CSRF Token) ===== -->
<form method="POST" action="/change-password">
    <input type="hidden" name="csrf_token" value="{ csrf_token() }">
    <input type="hidden" name="username" value="admin">
    <input type="password" name="new_password">
    <button type="submit">修改密码</button>
</form>
```

7. 安全建议

7.1 CSRF 防御措施优先级

优先级	措施	说明
①	CSRF Token	所有 POST/PUT/DELETE 请求必须携带随机 Token
②	SameSite Cookie	设置 SESSION_COOKIE_SAMESITE='Lax' 或 'Strict'
③	验证 Referer/Origin	检查请求来源是否合法 (辅助防御)
④	关键操作二次确认	修改密码需要验证原密码

7.2 Flask 开发者 CSRF 检查清单

- ☐ 是否安装了 Flask-WTF 或 Flask-SeaSurf?
- ☐ 是否启用了全局 CSRF 保护?

- ☐ 所有 POST 表单是否都包含了 {{ csrf_token() }}?
- ☐ AJAX 请求是否在请求头中携带了 CSRF Token?
- ☐ 关键操作（修改密码、转账）是否有额外验证?
- ☐ Session Cookie 是否设置了 SameSite='Lax'?
- ☐ 是否有豁免 CSRF 的接口（如 Webhook）? 如有，是否有其他验证?

7.3 关键操作额外验证

对于高风险操作（修改密码、转账、删除账户），除了 CSRF Token 外，还应：

额外验证	说明
验证原密码	修改密码时必须输入原密码
短信/邮箱验证码	高风险操作发送验证码确认
操作确认弹窗	前端确认”是否执行此操作”
二次授权	需要更高权限用户审批

7.4 完整的 CSRF 防御架构



附录：OWASP CSRF 防御指南

OWASP 关于 CSRF 的推荐做法：

1. **使用 Anti-CSRF Token**：每个会话生成唯一的 Token，包含在表单中或请求头中
2. **SameSite Cookie**：设置 SameSite=Strict 或 SameSite=Lax
3. **验证标准请求头**：检查 Origin 和 Referer 头
4. **关键操作重新认证**：修改密码等操作要求输入原密码

5. **使用自定义请求头：**AJAX 请求使用自定义头 (X-Requested-With)，因为跨域请求无法设置自定义头

相关漏洞类型： - CWE-352: 跨站请求伪造 (Cross-Site Request Forgery) - 与 XSS 的区别: CSRF 利用的是用户的身份认证, XSS 利用的是用户对网站的信任

报告编写： Claude Code AI Assistant

报告版本： v1.0

参考标准： OWASP Top 10 (2021) - A01:2021 Broken Access Control

测试环境： Flask 用户管理系统

免责声明： 本报告仅用于安全学习和教学目的